

LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 2: Neural Networks - The Foundation
Lecture	Lecture 2.3: Training Neural Networks

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Explain the overall training loop: forward pass, loss calculation, backpropagation, and weight update.
- Describe what a loss function is and give examples for classification and regression tasks.
- Articulate how backpropagation computes gradients to determine how each weight should be adjusted.
- Explain what gradient descent is and how the learning rate controls the training process.
- Identify common training problems — overfitting, underfitting, vanishing gradients — and describe how they are addressed.

Introduction

A freshly initialised neural network knows nothing. Its weights are set to random values, and if you ask it to perform any task, it will produce essentially random outputs. Training is the process of transforming this random system into a capable one — through a systematic, iterative procedure of showing examples, measuring errors, and adjusting weights to reduce those errors. Understanding training is the key to understanding why AI systems require so much data and compute, why they can fail in predictable ways, and what trade-offs are involved in building them.

The training process combines three fundamental ideas: a loss function that quantifies how wrong the model is, backpropagation that efficiently computes how each weight contributed to the error, and gradient descent that adjusts weights in the direction that reduces error. These three components — working together in millions or billions of iterations — are what allows a randomly initialised network to eventually write code, generate images, or answer questions in natural language.

For professionals making decisions about AI systems, understanding training clarifies why more data generally improves models (more training examples means more weight adjustment opportunities), why training is expensive (billions of weight adjustments require enormous compute), and why training data quality matters so critically (bad examples teach bad weight adjustments). The training process is where the model's character — its capabilities, limitations, and biases — is fundamentally shaped.

Core Concepts Explained

The Training Loop

Definition: The training loop is the iterative cycle of: (1) forward propagation to generate a prediction from input data, (2) loss calculation to measure prediction error, (3) backpropagation to compute how each weight

contributed to the error, and (4) weight update to reduce the error. This loop repeats for thousands or millions of training examples.

Business relevance: The training loop is why AI model development takes time and compute resources. A typical large language model training run involves trillions of training loop iterations across massive computing clusters running for weeks or months. This cost is why pre-trained foundation models are so valuable — they represent enormous training investments that can be shared through fine-tuning.

Practitioner relevance: Understanding the training loop helps professionals ask the right questions when evaluating AI development proposals: How many training examples are available? How many training iterations are planned? What is the compute budget? These questions directly determine what quality of model is achievable.

Real-world example: Training GPT-3 on 300 billion tokens involved billions of training loop iterations, processing each token in context, computing loss against the next token, backpropagating gradients, and updating 175 billion parameters. The total compute cost was estimated at millions of dollars in cloud compute — which is why accessing GPT-3 via API (rather than retraining it) was the economically rational choice for most organisations.

Loss Functions

Definition: A loss function (also called a cost function or objective function) is a mathematical measure of how different the model's predictions are from the correct answers. Common loss functions include cross-entropy loss (for classification and language modelling) and mean squared error (for regression). The goal of training is to minimise the loss.

Business relevance: The choice of loss function determines what the model is optimised to do. A model trained to minimise cross-entropy loss on next-token prediction learns to predict probable text. A model trained with RLHF uses human preference scores as part of its loss signal, learning to produce outputs humans rate highly. The loss function is where business objectives are translated into mathematical training signals.

Practitioner relevance: Misalignment between the loss function and the actual business objective is a common source of AI system failures. A model trained to minimise false negatives in fraud detection will behave differently from one trained to minimise overall error rate. Defining the right loss function is as important as choosing the right architecture.

Real-world example: A medical AI trained to maximise overall accuracy on a dataset with 95% negative (no disease) cases and 5% positive (disease) cases can achieve 95% accuracy by predicting negative for every case. This appears to be high accuracy but is clinically useless. The loss function must penalise false negatives much more heavily than false positives for a medical screening tool.

Risks or limitations: Goodhart's Law applies directly: when a measure (the loss function) becomes the target, it ceases to be a good measure of the actual objective. Models will find ways to minimise the specified loss that may not align with the intended outcome.

Backpropagation

Definition: Backpropagation is the algorithm that efficiently computes the gradient of the loss function with respect to every weight and bias in the network. It works by propagating the error signal backwards through the network, using the chain rule of calculus to determine each parameter's contribution to the total error.

Business relevance: Backpropagation is the computational engine that makes training deep networks practically feasible. Without it, computing gradients for a 175-billion-parameter model would be computationally intractable. Backpropagation's efficiency is the reason that GPU clusters can train large models in weeks rather than years.

Practitioner relevance: Understanding that backpropagation computes gradients — not explicit rules about what to learn — helps explain why neural network training is an optimisation process rather than a knowledge-encoding process. The model is not being told what to learn; it is being pushed toward weights that reduce a numerical error signal.

Real-world example: When training a language model on the example The cat sat on the [mat], the model initially predicts a random distribution over the vocabulary. The cross-entropy loss is high. Backpropagation computes how much each of the 175 billion parameters contributed to this error. Weights associated with connections relevant to predicting words that follow cat sat on the receive the largest adjustments.

Risks or limitations: The vanishing gradient problem occurs in deep networks when gradients become exponentially smaller as they propagate backwards through many layers, making early layer weights essentially unable to learn. This is addressed by better activation functions (ReLU, GELU), better weight initialisation, and architectural innovations like residual connections.

Gradient Descent and Learning Rate

Definition: Gradient descent is the optimisation algorithm that uses the gradients computed by backpropagation to update weights in the direction that reduces loss. The learning rate is a hyperparameter that controls how large each weight update step is.

Business relevance: The learning rate is one of the most important hyperparameters in training. Too high: training is unstable and weights overshoot the optimal values. Too low: training is slow and may get stuck in suboptimal solutions. Learning rate scheduling — adjusting the rate during training — is standard practice in large model training.

Practitioner relevance: When a vendor or engineering team says they are training a model and encountering instability or slow convergence, the learning rate and its schedule are often the first parameters to investigate. This is a fundamental tuning problem, not a fundamental architectural flaw.

Real-world example: Modern large language model training typically uses a cosine learning rate schedule: the learning rate starts low (warm-up period), rises to a peak, then gradually decreases over training. This schedule reduces the risk of early instability and allows fine-grained refinement as training progresses — analogous to taking large steps when far from the destination and small steps when close.

Risks or limitations: Learning rate is one of the most sensitive hyperparameters. Small changes can dramatically affect training stability and final model quality. Most large model training runs use adaptive learning rate algorithms (Adam, AdamW) that automatically adjust the learning rate per parameter, reducing but not eliminating this sensitivity.

Practical Applications

Evaluating AI Training Data Requirements

The training loop runs once per training example (or per mini-batch). More diverse, high-quality training examples lead to better gradient signals and ultimately better weights. For any AI project, the training data inventory — quantity, quality, coverage, and labelling accuracy — should be evaluated as rigorously as the model architecture.

Understanding Why Fine-tuning is Cost-effective

Fine-tuning starts from pre-trained weights (already optimised through billions of training loop iterations) and runs additional training loop iterations on a smaller, domain-specific dataset. Because the pre-trained weights

already capture general knowledge, far fewer fine-tuning iterations are needed — dramatically reducing training cost.

Recognising Overfitting in Deployed Models

Overfitting occurs when a model performs well on training data but poorly on new data. Signs in production: a model is accurate on data similar to training data but fails on variations or edge cases. Mitigation involves more diverse training data, regularisation techniques, or model architecture changes.

Best Practices

- Use validation sets to detect overfitting during training. Monitor loss on a held-out validation set during training. If training loss continues to decrease but validation loss stops improving or increases, the model is overfitting. Stop training or apply regularisation.
- Apply learning rate scheduling. A fixed learning rate rarely produces optimal results. Cosine scheduling, step decay, or adaptive algorithms like Adam are standard practice. Consult AI engineering teams about scheduling strategy when overseeing training projects.
- Ensure training data covers edge cases. Models generalise from training distribution to deployment distribution. If training data does not include edge cases or rare scenarios, the model will fail on those inputs in production. Explicitly include edge cases and rare but important examples.
- Match the loss function to the actual business objective. Translating business goals into precise mathematical loss functions is a critical design step. Misalignment between the loss function and the intended outcome is one of the most common sources of AI system failure in practice.

Common Mistakes and Misconceptions

Misconception: Assuming training accuracy directly reflects deployment performance.

Correct approach: A model that achieves very high training accuracy may be overfitting — memorising training examples rather than learning generalisable patterns. Validation and test set evaluation is mandatory before drawing conclusions about deployment performance.

Misconception: Treating the training process as a black box that engineers manage.

Correct approach: Business decisions profoundly affect training outcomes: what data is collected, how labels are defined, what loss function is used, and what evaluation metrics are tracked. Business stakeholders have direct influence over — and responsibility for — these decisions.

Misconception: Believing that more training always improves the model.

Correct approach: Training a model for too long on insufficient data leads to overfitting. Training must be stopped at the point of optimal generalisation, which requires monitoring validation performance, not simply maximising training time.

Evaluation Checklist: AI Training Quality Checklist — Business Stakeholder Review

Use this checklist when reviewing an AI training project to ensure that business objectives are correctly translated into training decisions.

- ☐ The business objective has been precisely defined before the loss function was chosen.
- ☐ The training dataset covers the range of inputs the model will encounter in production.
- ☐ Class imbalances (e.g., rare events) have been addressed in the training data or loss function.
- ☐ A separate validation set (not used for training) is being monitored during training.
- ☐ A separate test set (not used for training or validation) will be used for final performance evaluation.
- ☐ The evaluation metrics reported map directly to the business objective, not just statistical performance.
- ☐ Edge cases and failure scenarios have been deliberately included in training and evaluation data.
- ☐ Overfitting has been checked: training and validation performance converge appropriately.
- ☐ The training data has been reviewed for labelling quality and potential biases.
- ☐ Plans for monitoring model performance post-deployment are in place.

How to use this: *This checklist is intended for business stakeholders, not engineering teams. Each item represents a business decision that requires stakeholder input, even if the technical implementation is delegated to engineers.*

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. A model achieves 99% accuracy on its training dataset but only 65% accuracy on the test dataset. What is the most likely cause?

- A. The test dataset is too large.
- B. The loss function was not defined correctly.
- C. Overfitting — the model has memorised training examples rather than learning generalisable patterns.
- D. The learning rate was too low.

Answer: C. Overfitting — the model has memorised training examples rather than learning generalisable patterns.

Why: *Large gaps between training and test performance indicate overfitting — the model has learned the specific training examples rather than the underlying patterns that generalise to new data.*

2. A healthcare company trains an AI model to maximise overall diagnostic accuracy on a dataset that is 95% negative cases and 5% positive cases. The model consistently predicts negative for all cases. What went wrong?

- A. The model architecture has too few layers.
- B. The loss function was misaligned with the actual clinical objective — maximising overall accuracy on imbalanced data does not require the model to learn to detect the positive cases.
- C. Backpropagation failed to compute correct gradients.
- D. The training dataset was too large.

Answer: B. The loss function was misaligned with the actual clinical objective — maximising overall accuracy on imbalanced data does not require the model to learn to detect the positive cases.

Why: This is the class imbalance problem combined with loss function misalignment. A model that always predicts the majority class achieves high accuracy without any learning. The loss function must weight false negatives appropriately.

3. Backpropagation computes which quantity for each weight in a neural network?

- A. The optimal weight value that minimises the loss
- B. The gradient — the direction and magnitude of change in loss with respect to that weight
- C. The weight's current activation value
- D. The number of training examples that activated that weight

Answer: B. The gradient — the direction and magnitude of change in loss with respect to that weight

Why: Backpropagation uses the chain rule of calculus to compute the gradient of the loss with respect to every weight. Gradient descent then updates weights in the direction that reduces the loss.

4. Why is fine-tuning a pre-trained model more cost-effective than training a model from scratch for a specific domain task?

- A. Fine-tuned models have fewer parameters and are cheaper to run.
- B. Pre-trained weights already encode general knowledge from billions of training iterations, so far fewer fine-tuning iterations are needed to specialise the model.
- C. Fine-tuning does not require labelled training data.
- D. Fine-tuning avoids the need to define a loss function.

Answer: B. Pre-trained weights already encode general knowledge from billions of training iterations, so far fewer fine-tuning iterations are needed to specialise the model.

Why: Fine-tuning starts from a strong initialisation (pre-trained weights) rather than random initialisation. The pre-trained weights already capture useful patterns, so adaptation to a new domain requires far fewer training iterations.

Reflection Questions

1. The training loop is fundamentally an error-correction process — measure mistakes, trace responsibility, adjust behaviour. What analogies does this have to organisational learning and continuous improvement processes you are familiar with?
2. Loss function design requires translating a business objective into a precise mathematical measure. Think of a business outcome you care about — how would you attempt to quantify it as a mathematical measure that a model could optimise?
3. Overfitting in ML — performing well on training data but poorly on new data — has analogues in human learning and organisational processes. Can you identify examples where organisations over-optimised for measured performance at the expense of genuine capability?
4. The cost of training large models is a significant barrier to entry. What are the implications of this cost barrier for competition in AI, access to AI capabilities, and concentration of AI development power?
5. Given that training data quality profoundly shapes model capabilities and biases, who in your organisation should be responsible for decisions about what data is used to train AI systems that affect your customers?

Key Takeaways

- The training loop — forward pass, loss calculation, backpropagation, weight update — is the fundamental mechanism through which neural networks learn from data.
- Loss functions translate business objectives into mathematical training signals; misaligned loss functions are a common source of AI system failures.
- Backpropagation efficiently computes how each weight contributed to the prediction error, enabling gradient descent to adjust weights at scale.
- The learning rate controls the size of weight adjustment steps — too high causes instability, too low causes slow convergence or poor solutions.
- Overfitting occurs when a model memorises training data rather than learning generalisable patterns; validation monitoring and regularisation are the primary mitigations.
- Training data quality — coverage, labelling accuracy, balance, and representativeness — directly shapes the model's capabilities and biases.
- Fine-tuning pre-trained models is more cost-effective than training from scratch because pre-trained weights provide a strong starting point encoding general knowledge.

Recommended Next Actions

6. Draw the training loop as a diagram: forward pass, loss, backpropagation, weight update. Annotate each step with one sentence explaining what happens and why.
7. For one AI system in your organisation or industry, identify: What is the loss function likely optimising for? Is that aligned with the actual business objective? What potential misalignment exists?
8. Add to your glossary: Loss Function, Cross-Entropy Loss, Backpropagation, Gradient Descent, Learning Rate, Overfitting, Underfitting, Regularisation.
9. Write a brief stakeholder briefing (3 to 4 sentences) explaining why training data quality is a business responsibility, not just a technical one.